

Deployment & Launch Plan

1. Purpose

This document defines how the Sanatan Dharma platform should move from development to a safe, reliable public launch.

The launch process should ensure:

- Stable hosting
- Secure production configuration
- Reliable database access
- Backups
- Monitoring
- Analytics
- SEO readiness
- Legal readiness
- Content readiness
- AI readiness
- Rollback capability
- Post-launch monitoring

The goal is to avoid treating deployment as simply “click publish.”

2. Recommended Production Architecture

Recommended V1 architecture:

```
Users
↓
Custom Domain
↓
Vercel
↓
Next.js Application
↓
Supabase
├─ PostgreSQL
├─ Authentication
├─ Storage
└─ pgvector
```

↓
AI Provider

Supporting services may include:

- Error monitoring
- Analytics
- Email provider
- DNS provider
- GitHub

3. Deployment Environments

Use three separate environments:

Development

Used for local development.

Example:

```
localhost
```

Uses development credentials and development database where possible.

Staging

Used for:

- Full testing
- Content review
- AI testing
- Stakeholder review
- Pre-launch verification

Staging should behave as closely as possible to production.

Production

The live public website.

Only tested and approved code should reach production.

4. Environment Separation

Development and production should not share important credentials.

Separate:

- Database credentials
- Authentication configuration
- Storage configuration
- AI API keys
- Monitoring configuration
- Analytics
- Application URLs

This reduces the risk of accidental production changes.

5. Source Control

Recommended:

GitHub

Main repository should contain:

- Application code
- Database migrations
- Tests
- Configuration templates
- Documentation
- Seed scripts

Never commit:

- API keys
 - Database passwords
 - private tokens
 - production secrets
-

6. Branching Strategy

For this project, keep branching simple.

Recommended:

```
main
feature/*
fix/*
```

`main` should always represent deployable code.

Every significant feature should use a separate branch.

Example:

```
feature/gita-reader
feature/search
feature/ask-dharma
fix/verse-navigation
```

7. Preview Deployments

Each pull request or feature branch should receive a preview deployment where possible.

Recommended flow:

```
Code change
↓
GitHub
↓
Vercel Preview
↓
Review
↓
Tests pass
↓
Merge to main
```

↓

Production deployment

This makes it easier to catch design and functional problems before release.

8. CI Checks

Before merging to production, automatically run:

- TypeScript check
- ESLint
- Unit tests
- Build
- Critical integration tests

Later add:

- Playwright
 - Accessibility checks
 - Security scanning
-

9. Production Hosting

Recommended:

Vercel

Why:

- Strong Next.js integration
- Preview deployments
- HTTPS
- CDN
- Server functions
- Simple Git integration

The application should not depend on proprietary hosting behavior unnecessarily.

10. Database Hosting

Recommended:

Supabase PostgreSQL

Production database should be a separate project/environment from development.

Enable:

- Secure passwords
 - Row Level Security
 - Backups
 - Restricted administrative access
-

11. Database Migration Process

Production database changes should happen only through migrations.

Recommended flow:

```
Develop schema change
↓
Create migration
↓
Test locally
↓
Test in staging
↓
Backup production
↓
Run migration
↓
Verify
↓
Deploy application
```

Avoid manual production schema editing.

12. Database Backup Strategy

Backups are especially important because verified scripture data and source metadata may require significant effort to build.

Production should have:

- Automated database backups
- Regular exported backups
- Periodic testing of restore procedures

Critical knowledge data may also be exported into versioned archival files.

13. Recovery Testing

Having backups is insufficient if they cannot be restored.

Periodically test:

- Database restore
- Content recovery
- Authentication recovery
- Storage recovery

Document the restoration process.

14. Storage

Recommended V1:

Supabase Storage

May contain:

- Images
- Source documents
- Profile images
- Future audio

Separate public and private storage buckets.

15. Public Storage

Suitable for:

- Public illustrations

- Approved deity images
 - Festival images
 - Public article images
-

16. Private Storage

Suitable for:

- Restricted source documents
- Copyrighted reference scans
- Internal reviewer material
- Administrative uploads

Users should not receive direct public URLs for restricted files.

17. Domain Strategy

Before public launch, select a strong product name and custom domain.

Domain qualities:

- Easy to remember
- Easy to spell
- Respectful
- Suitable for long-term branding
- Not too narrowly tied to one feature

Possible extensions:

```
.com  
.org  
.in
```

Availability should be checked before choosing the final name.

18. DNS Configuration

DNS should point the chosen domain to the production hosting environment.

Configure:

- Root domain
- `www`
- HTTPS
- Redirect strategy

Choose one canonical domain.

Example:

```
https://example.com
```

and redirect:

```
https://www.example.com
```

to the canonical version, or vice versa.

19. HTTPS

All production traffic must use HTTPS.

HTTP should automatically redirect to HTTPS.

No authentication, user data, or AI conversation should ever be transferred over insecure HTTP.

20. Environment Variables

Production secrets should be configured through hosting environment variables.

Examples:

```
DATABASE_URL  
SUPABASE_URL  
SUPABASE_ANON_KEY  
SUPABASE_SERVICE_ROLE_KEY
```

```
AI_API_KEY
SITE_URL
```

Never expose server-only secrets through client-side environment variables.

21. Secret Rotation

If a secret is accidentally exposed:

1. Revoke it immediately.
2. Create a new secret.
3. Update production configuration.
4. Review logs.
5. Determine whether unauthorized access occurred.

Do not simply delete the secret from Git and continue using it.

22. Authentication Production Configuration

Before launch verify:

- Production callback URLs
- Google login domains
- Email verification
- Password-reset URLs
- Cookie security
- Session expiry
- Logout behavior

Authentication bugs should block launch.

23. Google Login

If Google authentication is enabled:

Create production OAuth configuration separately.

Allowed redirect URLs should include only the required production/staging endpoints.

Avoid broad wildcard redirects.

24. Email

V1 may need transactional emails for:

- Email verification
- Password reset
- Security notifications

Choose a transactional email provider during implementation.

Marketing emails should remain separate and opt-in.

25. AI Production Configuration

Before Ask Dharma is enabled publicly:

- Set production API keys
- Set strict usage limits
- Configure fallback behavior
- Verify retrieval corpus
- Verify citation validation
- Configure monitoring
- Test provider failures

AI should be controlled by a feature flag.

26. AI Feature Flag

Recommended:

```
ASK_DHARMA_ENABLED=true/false
```

If serious problems occur, administrators should be able to disable Ask Dharma without taking down the rest of the site.

27. AI Emergency Disable

If the AI begins:

- Fabricating scriptures
- Returning unsafe answers
- Exposing restricted content
- Experiencing uncontrolled cost
- Suffering provider outages

disable AI temporarily while keeping core content available.

28. Search Production Readiness

Before launch:

- Rebuild search index
 - Verify aliases
 - Test Hindi
 - Test English
 - Test Sanskrit transliteration
 - Verify no draft content is indexed
-

29. AI Index Production Readiness

Only include:

- Approved content
- Verified content where required
- Legally usable material

Exclude:

- Drafts
 - Rejected sources
 - Archived content
 - Internal notes
 - Restricted full-text material
-

30. Content Freeze

Before the first major launch, introduce a short content freeze.

During the freeze:

- No major schema changes
- No large content imports
- Only critical corrections

This allows final testing against a stable version.

31. Launch Content Requirements

The platform should not launch with mostly empty pages.

Recommended launch baseline:

Learn

10–20 high-quality concepts.

Bhagavad Gita

Strong chapter and verse experience.

Scriptures

Major overview pages.

Deities

Approximately 7 strong pages.

Mantras

10–20 verified entries.

Festivals

10–15 major entries.

Exact numbers are less important than quality.

32. Homepage Content Review

Before launch, manually review every homepage element.

Verify:

- Headline
- Description
- CTA
- Featured content
- Daily shloka
- Festival
- Links
- Hindi version

The homepage is the highest-visibility page.

33. Source Verification Before Launch

For launch content confirm:

- Sources exist
- Links work
- Copyright status is known
- Translator/commentator attribution is correct
- Scripture references are correct

Any flagship page with questionable sourcing should remain unpublished.

34. AI Knowledge Coverage

Ask Dharma should not imply that it understands every aspect of Sanatan Dharma at launch.

When knowledge coverage is limited, the assistant should say so.

The UI may state:

Ask Dharma currently answers using the verified material available in our growing library.

35. Legal Pages

Before collecting user information, publish:

- Privacy Policy
- Terms of Service

Also recommended:

- Content Policy
 - Source Policy
 - Copyright Policy
 - AI Disclaimer
 - Contact / Report Error page
-

36. Privacy Policy

Should explain:

- What account information is collected
- Why it is collected
- Authentication providers
- Analytics
- AI conversation handling
- Cookies
- Data deletion
- Third-party services

Final legal text should be reviewed appropriately before serious commercial scale.

37. Terms of Service

Should address:

- Educational purpose
 - Acceptable use
 - Accounts
 - AI limitations
 - Intellectual property
 - Liability limitations
 - Prohibited abuse
-

38. Content Policy

Publicly explain:

- How sources are selected
- How traditions are represented
- How mistakes are corrected
- How AI differs from scripture
- How users report errors

This helps reinforce trust.

39. Copyright Review

Before launch inspect:

- Modern translations
- Commentaries
- Illustrations
- Deity artwork
- Photography
- Audio
- Source documents

Never assume content found online can be republished.

40. SEO Launch Checklist

Verify:

- Sitemap
 - Robots
 - Metadata
 - Canonical URLs
 - English/Hindi alternate URLs
 - Open Graph
 - Structured data
 - Correct page titles
 - Indexable public pages
-

41. Search Engine Indexing

Staging should normally be blocked from search indexing.

Production should become crawlable only when ready.

Avoid having unfinished staging pages appear in Google.

42. Analytics

Install analytics before or during launch.

Recommended metrics:

- Visitors
- Page views
- Returning users
- Search usage
- Learning activity
- AI usage
- Bookmark usage

Avoid invasive tracking.

43. Error Monitoring

Recommended:

Sentry or similar.

Monitor:

- Frontend crashes
 - Backend exceptions
 - API failures
 - Database errors
 - AI service failures
-

44. Uptime Monitoring

Use a monitoring service for critical pages such as:

- Homepage
- Search
- Bhagavad Gita
- API health endpoint

This helps detect downtime quickly.

45. Health Endpoint

Consider:

```
/api/health
```

It may verify basic application availability.

Do not expose sensitive system details.

46. Logging

Production should capture:

- Errors
- Critical system events
- Admin publishing actions
- AI provider failures

Avoid unnecessary storage of private user messages.

47. Performance Launch Check

Run performance tests on:

- Homepage
- Karma concept
- Gita 2.47

- Search
- Ask Dharma

Pay particular attention to mobile connections.

48. Image Optimization

Before launch:

- Compress images
- Resize oversized assets
- Use modern formats
- Lazy load where appropriate
- Add alt text

Large imagery should not make the spiritual experience slow.

49. Mobile Launch Check

Test the live production candidate on actual mobile devices if possible.

Focus on:

- Navigation
 - Scripture reading
 - Hindi
 - Ask Dharma
 - Search
 - Login
 - Bookmarking
-

50. Accessibility Launch Check

Verify:

- Keyboard navigation
- Visible focus states
- Screen-reader labels
- Heading structure
- Image alt text
- Contrast

- Zoom/text scaling

Accessibility defects affecting core content should be fixed before launch.

51. Browser Launch Check

Verify production on:

- Chrome
 - Safari
 - Firefox
 - Edge
 - iPhone Safari
 - Android Chrome
-

52. Pre-Launch Security Review

Verify:

- No secrets exposed
 - Admin routes protected
 - RLS enabled
 - User data isolated
 - File uploads validated
 - Rate limits enabled
 - AI usage limited
 - Authentication callbacks locked down
 - Restricted files protected
-

53. Admin Security Review

Test with a normal user manually attempting admin API calls.

Every unauthorized request must fail server-side.

Never assume hidden buttons provide security.

54. Launch Checklist

Before public release:

```
[ ] Production domain connected
[ ] HTTPS working
[ ] Database backed up
[ ] Migrations complete
[ ] Authentication verified
[ ] Search verified
[ ] English verified
[ ] Hindi verified
[ ] Bhagavad Gita verified
[ ] AI citations verified
[ ] AI rate limits enabled
[ ] Admin protected
[ ] Analytics enabled
[ ] Error monitoring enabled
[ ] Legal pages published
[ ] SEO complete
[ ] Sitemap submitted
[ ] Mobile tested
[ ] Browser tested
[ ] Critical QA passed
```

55. Soft Launch

Do not immediately promote the platform widely.

Recommended first release:

Soft Launch

Invite a small number of users.

Suggested groups:

- Beginners
- Hindi users
- English users
- Students
- Knowledgeable Sanatan Dharma readers

Collect feedback before broader promotion.

56. Soft Launch Goals

Look for:

- Confusing navigation
- Incorrect content
- Search failures
- AI failures
- Hindi issues
- Mobile problems
- Missing topics
- Trust concerns

These are easier to fix with a small user base.

57. Beta Status

The platform can initially display:

Beta

This communicates that the product is improving while still requiring a high standard for scripture accuracy.

Beta should not be used as an excuse for knowingly publishing unreliable religious information.

58. Public Launch Criteria

Move from soft launch to wider public launch when:

- Critical bugs are resolved
 - Search is reliable
 - Core Gita experience works
 - AI has acceptable evaluation results
 - Users understand source labels
 - Mobile experience is strong
 - Content workflow is functioning
-

59. Launch Announcement

Public launch messaging should emphasize the core value proposition:

Learn Sanatan Dharma through structured, accessible, source-backed knowledge in English and Hindi.

Avoid claiming:

“The ultimate authority on Sanatan Dharma.”

Trust should be earned rather than declared.

60. Post-Launch Monitoring

Monitor closely after release:

- Error rate
 - API failures
 - Login failures
 - Search failures
 - AI errors
 - Content reports
 - Performance
 - AI costs
-

61. Post-Launch Feedback

Provide clear feedback channels:

- Report content error
- Report AI answer
- General feedback
- Contact

Users should be able to report mistakes easily.

62. Content Correction Priority

Prioritize corrections in this order:

1. Wrong scripture text
 2. Wrong verse/source attribution
 3. Major theological misrepresentation
 4. AI hallucination
 5. Wrong translation
 6. Broken source
 7. Typographical mistakes
-

63. Incident Response

For serious production issues:

```
Detect
↓
Assess severity
↓
Contain
↓
Disable affected feature if necessary
↓
Fix
↓
Test
↓
Deploy
↓
Review cause
```

64. Rollback Strategy

Every production deployment should be reversible.

If a release breaks critical functionality:

- Roll back application version
- Restore database only if required

- Disable problematic feature with feature flag

Database migrations should be designed carefully because some cannot be trivially reversed.

65. AI Incident Example

If Ask Dharma starts fabricating verses:

1. Disable Ask Dharma.
2. Preserve core website access.
3. Review retrieval and prompts.
4. Run golden evaluation tests.
5. Fix.
6. Re-enable after validation.

Do not keep unreliable AI online solely to preserve feature availability.

66. Content Incident Example

If a scripture text is found incorrect:

1. Mark affected content as needing review.
 2. Correct verified database record.
 3. Update translation/commentary relations if required.
 4. Rebuild search index.
 5. Rebuild AI knowledge chunk.
 6. Clear cache.
 7. Document revision.
-

67. Database Incident

If data corruption occurs:

- Stop writes if required
 - Identify affected records
 - Restore from backup
 - Reapply verified changes
 - Investigate root cause
-

68. Cost Monitoring

Track production expenses for:

- Hosting
- Database
- Storage
- AI generation
- Embeddings
- Monitoring
- Email
- Domain

AI will likely become one of the most variable costs.

69. AI Cost Alerts

Set alerts or internal thresholds for unexpected increases in:

- Requests
- Tokens
- Embedding generation
- Anonymous usage

Rate limits should prevent major abuse.

70. Scaling Strategy

Do not scale infrastructure prematurely.

When usage grows, review:

- Database load
- Search performance
- AI usage
- Storage
- CDN traffic

Scale whichever component is actually constrained.

71. Database Scaling

Potential future options:

- Larger Supabase instance
- Read replicas
- Query optimization
- Better indexing
- Connection pooling

Only add complexity when monitoring demonstrates a need.

72. Search Scaling

If PostgreSQL search becomes insufficient:

Evaluate:

- Typesense
- Meilisearch
- Algolia
- OpenSearch

Migration should use the existing search abstraction rather than rewriting the entire frontend.

73. AI Scaling

Possible future improvements:

- Semantic caching
- Smaller models for easy questions
- More efficient embeddings
- Better reranking
- Response streaming
- Provider routing

Cost and quality should both drive decisions.

74. Content Scaling

As the library expands, editorial workflow becomes more important than infrastructure.

Potential future teams:

- Sanskrit reviewers
- Hindi editors
- Researchers
- Tradition specialists
- Content administrators

The admin platform should support this growth.

75. Future Mobile Launch

Do not immediately duplicate web launch planning for apps.

Before mobile development:

- Stabilize APIs
- Understand user behavior
- Determine highest-value mobile features

Then create separate Android/iOS launch requirements.

76. Recommended Initial Public Release

Version 1 should communicate:

Focused, accurate, source-backed learning platform

rather than:

Every aspect of Sanatan Dharma in one app.

The narrower promise is easier to deliver well.

77. Success Signals After Launch

Positive early signals:

- Users finish beginner lessons
- Users return to read more
- Users explore sources

- Users search successfully
 - Ask Dharma citations are clicked
 - Users bookmark verses
 - Hindi users actively use the site
 - Low serious content-error rate
-

78. Product Review After Launch

After meaningful real usage, review:

- Which features people actually use
- Which pages bring search traffic
- Common AI questions
- Search queries with no results
- Most reported content
- Drop-off points
- User requests

Future development should be driven by this evidence.

79. First Post-Launch Priorities

Likely priorities:

- Fix bugs
- Improve source quality
- Expand useful concepts
- Improve search
- Improve AI grounding
- Improve Hindi content
- Expand Gita coverage

Avoid immediately adding 20 unrelated features.

80. Final Launch Principle

The platform should launch when its **core promise is trustworthy**, not when every planned feature exists.

The core promise is:

A user can learn about Sanatan Dharma in a clear, respectful, bilingual, and source-backed way, and can see where important information comes from.

81. Status

Deployment & Launch Plan:

Initial Version Complete

Major areas defined:

- Hosting
- Environments
- Git workflow
- CI
- Database deployment
- Backups
- Storage
- Domain
- HTTPS
- Authentication
- AI production setup
- Search indexing
- Content readiness
- Legal pages
- SEO
- Analytics
- Monitoring
- Security
- Soft launch
- Incident response
- Rollback
- Scaling
- Post-launch strategy